

↳ How 3D scenes onto 2D images

Lecture 2.1: Primitives & Transformations

↳ Introduction of points, lines & planes

↳ Most basic transformations for both 2D & 3D

2D Points:

↳ Written in inhomogeneous coordinates as:

$$x = \begin{pmatrix} x \\ y \end{pmatrix} \in \mathbb{R}^2$$

↳ In homogeneous coordinates,

$$\vec{x} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \in \mathbb{P}^2$$

↳ Extending dimensionality of vector space by one

↳ Where $\mathbb{P}^2 = \mathbb{R}^3 \setminus \{0, 0, 0\}$ called projective space

↳ Remove 0,0,0 coordinate

↳ $\mathbb{N}$ denotes homogeneous coordinates

↳ Homogeneous vectors that only differ by scale are considered equivalent

$$\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \\ 2 \end{pmatrix}$$

↳ Defined only up to scale

↳ Representing equivalence class

inhomogeneous to homogeneous vectors:

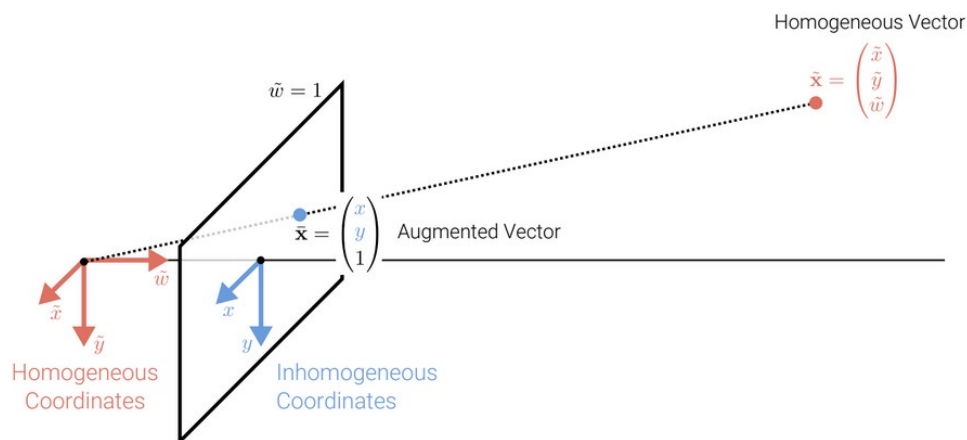
$$\vec{x} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \bar{x}$$

also called augmented vector $\bar{x}$

Convert homogeneous to inhomogeneous

$$\bar{x} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ \tilde{w} \end{pmatrix} = \frac{1}{\tilde{w}} \bar{x} = \frac{1}{\tilde{w}} \begin{pmatrix} \tilde{w}x \\ \tilde{w}y \\ \tilde{w} \end{pmatrix} = \begin{pmatrix} \tilde{w}x/\tilde{w} \\ \tilde{w}y/\tilde{w} \\ 1 \end{pmatrix}$$

Homogeneous points whose last element $\tilde{w}=0$ are called ideal points or points at infinity. These can't be represented by inhomogeneous coordinates.



2D lines:

Can be represented using homogeneous coordinates $\tilde{l} = (a, b, c)^T$

$$(\bar{x} \mid \tilde{l} \bar{x} = 0) \longleftrightarrow \{x, y \mid ax + by + c = 0\}$$

equal to zero
the points are
located on the line.

We can normalize $\tilde{l}$, so $\tilde{l} = (n_x, n_y, d)^T = (n, d)^T$ with $\|n\|_2 = 1$. In this case, n is the normal vector perpendicular to the line and d is its distance to the origin.

An exception is the line at infinity $\tilde{l}_\infty = (0, 0, 1)^T$ which passes through all ideal points

2D line arithmetic

In homogeneous coordinates, the intersection of two lines is given by

$$\tilde{x} = \tilde{l}_1 \times \tilde{l}_2$$

cross product
↑
x

↳ The line joining two points:

$$\tilde{l} = \bar{x}_1 \times \bar{x}_2$$

Proof:

if $x=2$ & $y=1$

$$\tilde{l}_1 = (0 \ 1 \ -1) \ , \ \bar{x}_1 = (x \ y \ 1)$$

$$\tilde{l}_1 \times \bar{x}_1 = 0$$

$$\tilde{l}_2 = (1 \ 0 \ -2) \ , \ \bar{x}_2 = (x \ y \ 1)$$

$$\tilde{l}_2 \times \bar{x}_2 = 0$$

$$[\tilde{l}_1] \times \tilde{l}_2 = \bar{x}$$

$$\begin{bmatrix} 0 & -1 & -1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} \times \begin{pmatrix} 1 \\ 0 \\ -2 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix}$$

NOTE: $[X]_X = \begin{bmatrix} 0 & z & -y \\ -z & 0 & x \\ y & -x & 0 \end{bmatrix}$
 ↳ Skew symmetric

if $\bar{x}_1 = (2 \ 0 \ 1)$

$\bar{x}_2 = (0 \ 1 \ 1)$

$$\begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 2 \\ 0 & -2 & 0 \end{bmatrix} \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix}$$

=

$$\begin{pmatrix} 1 \\ 2 \\ -2 \end{pmatrix}$$

2D conics:

$$\{ \bar{x} \mid \bar{x}^T Q \bar{x} = 0 \}$$

↳ Orientation of the plane

3D points:

↳ 3D points in inhomogeneous coordinates

$$x = \begin{pmatrix} x \\ y \\ z \end{pmatrix} \in \mathbb{R}^3$$

↳ In homogeneous coordinates

$$\tilde{x} = \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \in \mathbb{P}^3$$

with projective space $P^3 = \mathbb{R}^4 \setminus \{0,0,0,0\}$

↳ We have to define direction, so $\tilde{\omega} = 0$ but $\tilde{x}, \tilde{y}, \tilde{z} \neq 0$

↳ This space emphasizes the relation b/w pointed lines.

↳ It focuses on meaningful direction.

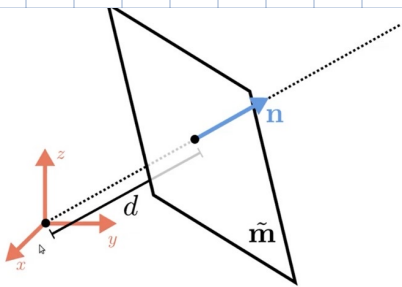
3D planes:

↳ In homogeneous coordinate:

$$\{\tilde{x} \mid \tilde{m}^T \tilde{x} = 0\} \longleftrightarrow \{x, y, z \mid ax + by + cz + d = 0\}$$

↳ We can normalize $\tilde{l}$, so $\tilde{l} = (n_x, n_y, n_z, d)^T = (n, d)^T$ with $\|n\|_2 = 1$. In this case, n is the normal vector perpendicular to the line and d is its distance to the origin.

↳ An exception is the plane at infinity $\tilde{l}_\infty = (0, 0, 0, 1)^T$ which passes through all ideal points.



$$\underbrace{(n_x \quad n_y \quad n_z \quad d)}_{\tilde{m}^T} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = 0$$

3D lines:

↳ Express a linear combination as two points p, q :

$$\{x \mid x = (1 - \lambda)p + \lambda q \wedge \lambda \in \mathbb{R}\}$$

↳ $\lambda = 0; x = p$

$\lambda = 1; x = q$

↳ $0 < \lambda < 1$, points on line $p-q$

↳ Parametres: $6 \rightarrow p: (x_1, y_1, z_1) \quad q: (x_2, y_2, z_2)$

↳ Degrees of freedom: 4 $\rightarrow$ no. of parameters to specify pos & orientation of an object

↳ $(1,0,0), (2,0,0) \rightarrow$ both point in same direction. Therefore, direction is redundant. So, 3 parameters for a point + 1 for direction = 4

3D Quadrics:

↳ 3D analog of 2D conics is a quadric surface

$$\{ \bar{x} \mid \bar{x}^T Q \bar{x} = 0 \}$$

↳ shape of Quadric

2D Transformations:

$$x' = x + t \longleftrightarrow \bar{x}' = \begin{bmatrix} I & t \\ 0 & 1 \end{bmatrix} \bar{x}$$

↳ Second expression allows to chain/invert translations

↳ 2 Degrees of freedom

Euclidean: (2D translation + 2D rotations, 3 DoF)

$$x' = Rx + t \longleftrightarrow \bar{x}' = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \bar{x}$$

↳ $R \in SO(2)$ is a special orthonormal rotation matrix $RR^T = I$ & $\det(R) = 1$

↳ These transformations preserve Euclidean distance.

↳ Distance b/w two points remain same before & after translation

Similarity: (2D Translation + Scaled 2D rotation, 4 DoF)

$$\hookrightarrow x' = sRx + t \longleftrightarrow \bar{x}' = \begin{bmatrix} sR & t \\ 0^T & 1 \end{bmatrix} \bar{x}$$

↳ $R \in SO(2)$ is a rotation matrix & s is arbitrary scale.

↳ Doesn't preserve distances but angle b/w lines.

Affine: (2D linear transformation, 6 DoF)

$$x' = Ax + t \longleftrightarrow x' = \begin{bmatrix} A & t \\ 0^T & 1 \end{bmatrix} \bar{x} \rightarrow \begin{bmatrix} A_{00} & A_{01} & b_0 \\ A_{10} & A_{11} & b_1 \\ 0 & 0 & 1 \end{bmatrix}$$

↳ $A \in \mathbb{R}^{2 \times 2}$

↳ Parallel lines remain parallel

Perspective (Homography, 8 DoF)

$$\tilde{x}' = \tilde{H} \tilde{x}$$

$$\left(\tilde{x} = \frac{1}{w} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \right)$$

$$\rightarrow \begin{bmatrix} A_{00} & A_{01} & b_0 \\ A_{10} & A_{11} & b_1 \\ h_0 & h_1 & 1 \end{bmatrix}$$

↳ Every coordinate of a square transforms to a different location

↳ $\tilde{H} \in \mathbb{R}^{3 \times 3}$ (specified up to scale) homogeneous

↳ Perspective transformation preserves straight lines.

↳ Lines NOT bending

2D Transformations on Co-vectors

↳ Considering any perspective 2D transformations

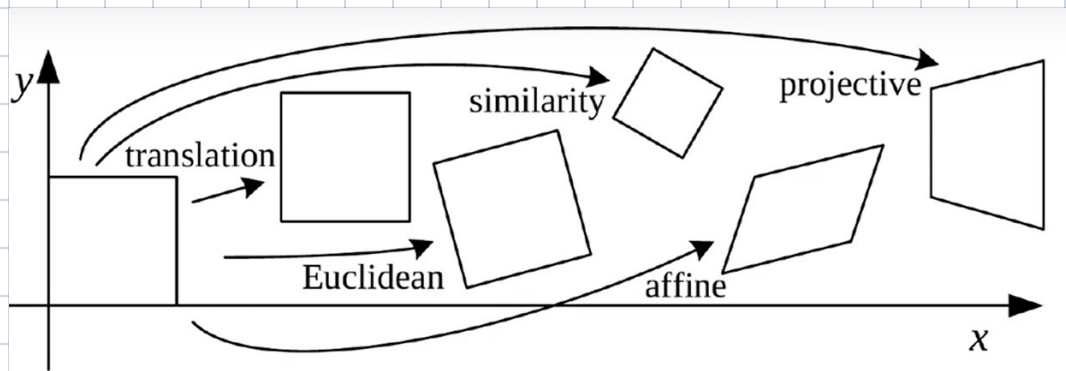
$$\tilde{x}' = \tilde{H} \tilde{x}$$

↳ Transformed 2D line equation is given by:

$$\tilde{l}'^T \tilde{x}' = \tilde{l}'^T \tilde{H} \tilde{x} = (\tilde{H}^T \tilde{l}')^T \tilde{x} = \tilde{l}^T \tilde{x} = 0$$

$$\tilde{l}' = \tilde{H}^{-1} \tilde{l}$$

expression of the line before the transformation



Overview of 2D transformation:

Transformation	Matrix	# DoF	Preserves	Icon
translation	$\begin{bmatrix} \mathbf{I} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	2	orientation	
rigid (Euclidean)	$\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	3	lengths	
similarity	$\begin{bmatrix} s\mathbf{R} & \mathbf{t} \end{bmatrix}_{2 \times 3}$	4	angles	
affine	$\begin{bmatrix} \mathbf{A} \end{bmatrix}_{2 \times 3}$	6	parallelism	
projective	$\begin{bmatrix} \tilde{H} \end{bmatrix}_{3 \times 3}$	8	straight lines	

"1" of the result of affine preserve

least restricted in a sense that it changes what others preserve

↳ Transformations form nested set of groups (closed under composition, inverse)

Projective is multiple affines



↳ projective is just affine without parallelism
Combine 2 affine transformations result is transformation of the same type

Apply transformation then inverse you get original back

↳ Interpret as restricted 3×3 matrices operating on 2D homogeneous coordinates.

Direct linear transformations for Homography Estimation: Estimate parameters from observations.

↳ Let $X = \{ \tilde{x}_i, \tilde{x}'_i \}$ denote a set of N 2D to 2D correspondence related by $\tilde{x}'_i = H \tilde{x}_i$.

After transformation

↳ Before transformation

↳ As correspondence vectors are homogeneous, they have same direction but differ in magnitude. Thus,

$$\tilde{x}'_i \times H \tilde{x}_i = 0$$

↳ Cross product is 0 if direction is same regardless of magnitude

$$\underbrace{\begin{bmatrix} 0^T & -\tilde{w}'_i \tilde{x}_i^T & \tilde{y}'_i \tilde{x}_i^T \\ \tilde{w}'_i \tilde{x}_i^T & 0^T & -\tilde{x}'_i \tilde{x}_i^T \\ -\tilde{y}'_i \tilde{x}_i^T & \tilde{x}_i \tilde{x}_i^T & 0^T \end{bmatrix}}_{A_i} \underbrace{\begin{bmatrix} \tilde{h}_1 \\ \tilde{h}_2 \\ \tilde{h}_3 \end{bmatrix}}_{\tilde{H}} = 0$$

NOTE:

- ↳ H has 8 unknowns
- ↳ To find 8 unknowns
- ↳ We need 8 equations
- ↳ One eq for 1 coordinate
- ↳ So, we need at least 4 pairs (x, y) to find the 8 knowns.

↳ Last scale is linearly dependent on first two & can be dropped.

↳ Therefore for each point we have two constraints.

↳ Therefore, $2N \times 9$

Examples

↳ H

↳ Each point correspondence yields two equations. Stacking all equations into $2N \times 9$ dimensional matrix A leads to following constrained least squares problem

$$\tilde{h}^* = \underset{\tilde{h}}{\operatorname{argmin}} \|A \tilde{h}\|_2^2 + \lambda (\|\tilde{h}\|_2^2 - 1)$$

minimize it as much as possible

$$= \underset{\tilde{h}}{\operatorname{argmin}} \tilde{h}^T A^T A \tilde{h} + \lambda (\tilde{h}^T \tilde{h} - 1)$$

As H is homogeneous we want $\|\tilde{h}\|_2$ must be 1

↳ fix $\|h\|_2 = 1$ as H is homogeneous

↳ solve using singular value decomposition.

↳ $A = UDV^T$.

↳ This resulting algorithm is called Direct Algorithm Transformation.



YouTube PK



Sign in



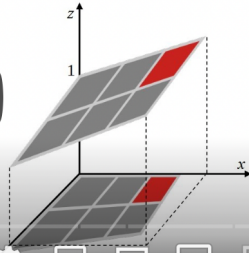
YouTube PK



Sign in

PROJECTIVE TRANSFORMATION

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} & b_0 \\ A_{10} & A_{11} & b_1 \\ h_0 & h_1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Projective Transformation



Computer Vision with Hüseyin Özdemir
2.16K subscribers

Subscribe



576



Share



Save



18K views 2 years ago #deeplearning #ai #aitutorial

Subscribe To My Channel https://www.youtube.com/@huseyin_ozde...

Video Contents: ...more

Computing Homography



Source Image

Destination Image

$\begin{bmatrix} x_d \\ y_d \\ 1 \end{bmatrix} = \begin{bmatrix} \tilde{x}_{s+1} & \tilde{h}_{s+1} & h_{s+1} & h_{s+1} & x_{s+1} \end{bmatrix}$
And then you take a third image that falls on plane 3.

How many unknowns? 9 ... But 8 degrees of freedom

Computing Homography | Image Stitching



First Principles of Computer Vision
77.3K subscribers

Subscribe



1.4K



Share



Save



96K views 4 years ago

First Principles of Computer Vision is a lecture series presented by Shree Nayar who is faculty in the Computer Science Department, School of Engineering and Applied Sciences, Columbia University. Computer Vision is the enterprise ...more

Lecture 2.2: Geometric Image Formation

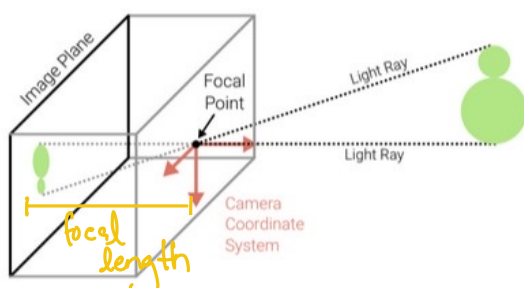
↳ How points on 2D space related to points on 3D space?

Origins of Pinhole:

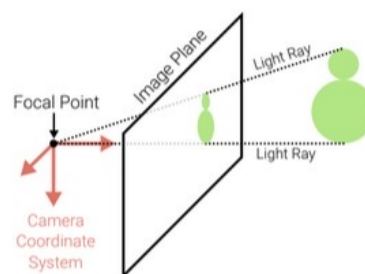
↳ light passes through a small hole
↳ Collected on the wall in front



Physical Camera Model



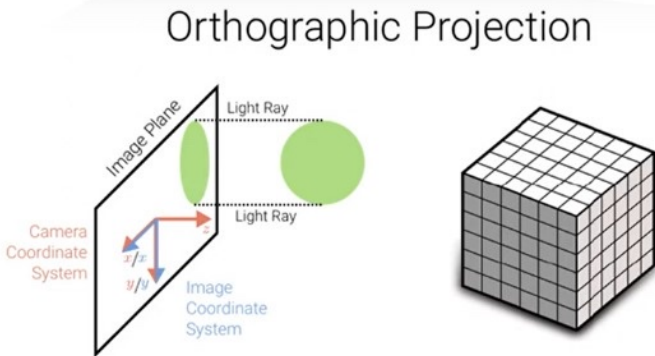
Mathematical Camera Model



Projection Models:

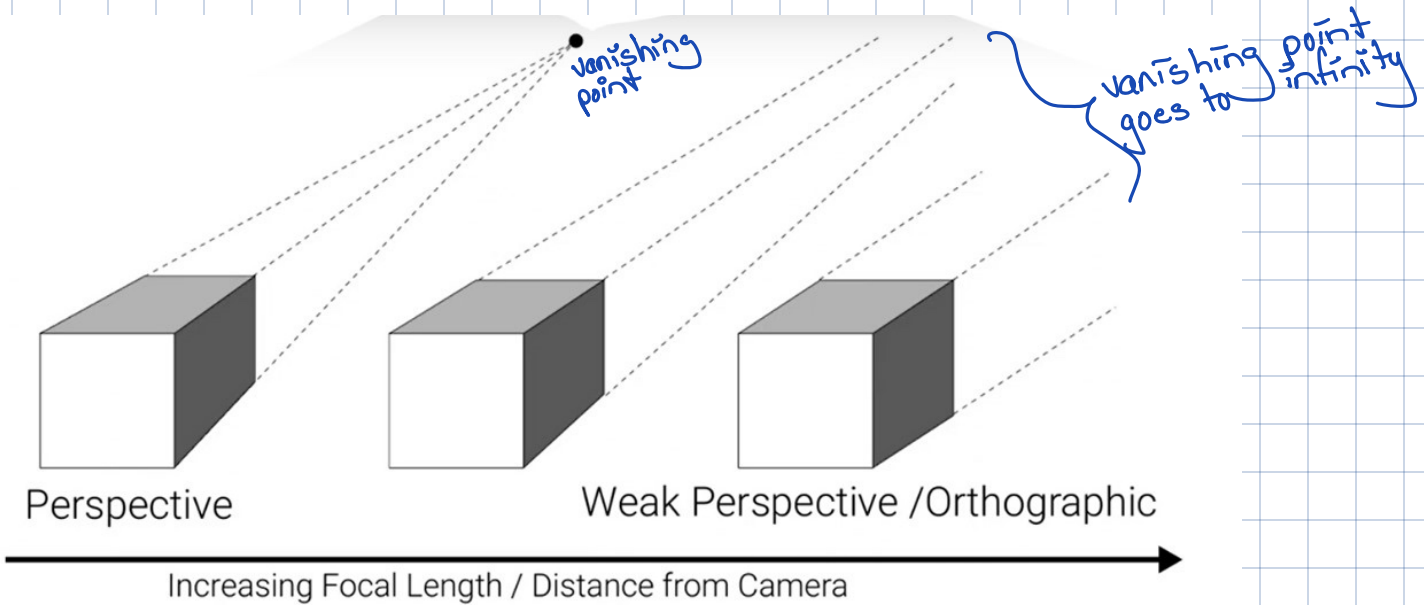
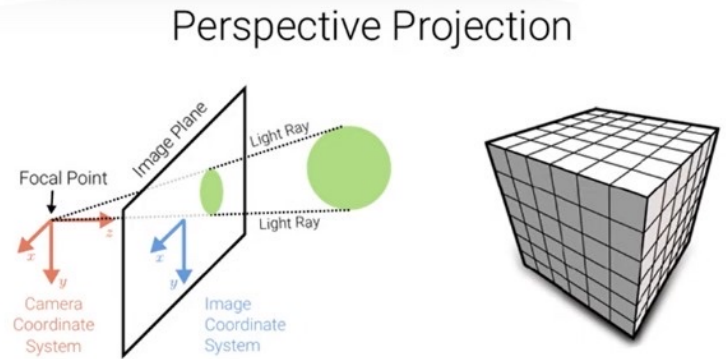
1) Orthographic Projection:

↳ light is assumed to travel parallel to each other's rays.



2) Perspective Projection:

↳ Pinhole model
 ↳ All light rays pass through that focal point.
 ↳ These rays are hitting the image plane at the intersection of that line with the image plane.



Orthographic projection: → Project object onto image plane from camera's perspective.



$$z_s = z_c$$

if $z_c = z_s$?

- ↳ x_s, y_s axes of image & x_c, y_c axes of camera are same.
- ↳ Light rays are parallel to z_c axis of the camera coordinate
- ↳ During project x, y remain while z is dropped.

$$x_s = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} x_c \longleftrightarrow \bar{x}_s = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \bar{x}_c$$

↓
 3d coordinates
 ↳ last element
 (z) is 0

↳ 4d coordinates
 ↳ last of $\bar{x}_c$ is 1
 so last of $\bar{x}_s$ is also 1.

- ↳ After projection, the distance of the 3D points from the image can't be recovered.

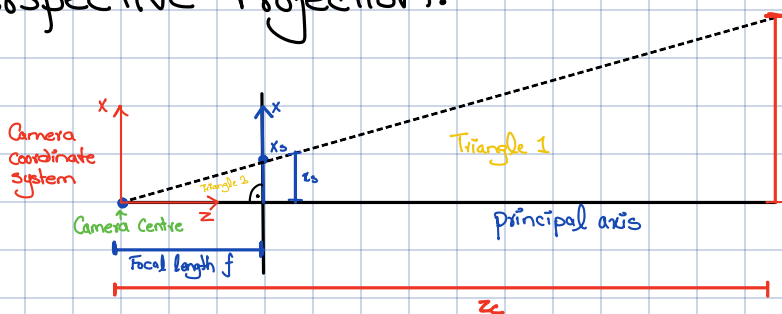
Scaled Orthographic Projections:

- ↳ Image sensor requires measurements in pixels, whereas previous approach uses metres.
- ↳ So, we scale them.

$$x_s = \begin{bmatrix} s & 0 & 0 \\ 0 & s & 0 \end{bmatrix} x_c \longleftrightarrow \bar{x}_s = \begin{bmatrix} s & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & s & 0 \\ 0 & 0 & 0 & s \end{bmatrix} \bar{x}_c$$

- ↳ Unit of s is pixels/m OR pixels/mm
- ↳ Structure & motion can be estimated using SVD.

Perspective Projection:



triangle 1 $\equiv$ triangle 2

$$\frac{x_s}{f} = \frac{z_c}{z_c}$$

- ↳ Pinhole projection formula.
- ↳ Applies to y_s also

- ↳ The light must pass through the pixel x_s , the camera centre, & the point x_c .
- ↳ Principal axis (orthogonal to the image plane) aligns with z -axis.
 ↳ Perpendicular
- ↳ We can formulate this mathematically in matrix multiplication, if we use homogeneous coordinates.

$$\begin{pmatrix} z_s \\ y_s \end{pmatrix} = \begin{pmatrix} f z_c / z_c \\ f y_c / z_c \end{pmatrix} \longleftrightarrow \bar{x}_s = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \bar{x}_c$$

↳ Projection matrix

↳ After projection, the distance of the 3D points from the image can't be recovered.

↳ Unit of 'f' is pixels.

NOTE:

↳ Optical axis is an imaginary line that runs through centre of the lens & represents ideal path for light rays.

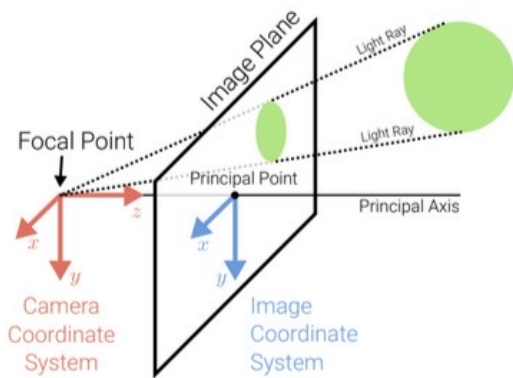
Principal point:

↳ It's inconvenient to store negative coordinates.

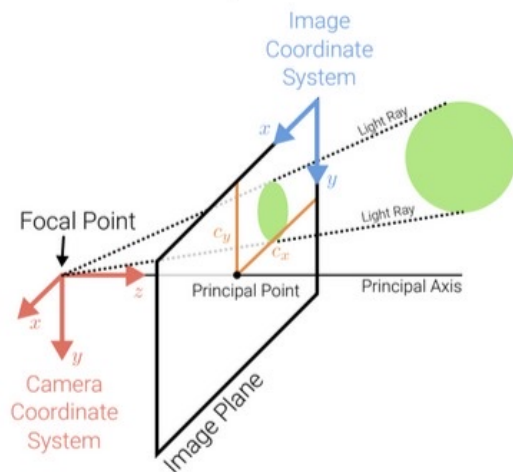
↳ So we move camera coordinates to top left to ensure all coordinates are positive.

↳ Also, principal point is added to define centre of image plane & correctly project 3D points onto 2D points.

Without Principal Point Offset



With Principal Point Offset



Complete perspective projection model without distortion:

$$\begin{pmatrix} z_s \\ y_s \end{pmatrix} = \begin{pmatrix} f_x z_c / z_c + s y_c / z_c + c_x \\ f_y y_c / z_c + c_y \end{pmatrix} \longleftrightarrow \bar{x}_s = \begin{bmatrix} f_x & s & c_x & 0 \\ 0 & f_y & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \bar{x}_c$$

↳ Focal length f_x differs from f_y

↳ Translate by c_x & c_y

↳ Added scaling factor that allows for modelling image sensors that are NOT perpendicular to the optical/principal axis.

↳ In practice, $f_x = f_y$, $s = 0$ but model $c = (c_x, c_y)$

↳ The 3×3 submatrix is called calibration matrix / Intrinsic matrix k .

Chaining transformations:

↳ Homogeneous coordinates allows us to chain transformations.

↳ If a image is represented in world coordinates we can chain
World $\rightarrow$ camera $\rightarrow$ image coordinates.

↳ let K be the calibration matrix (intrinsic) & $[R|t]$ the camera pose (extrinsics)

$$\bar{x}_s = [K \ 0] \bar{x}_c$$

$$= [K \ 0] \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \bar{x}_w$$

$$= \underset{3 \times 3}{K} \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \bar{x}_w$$

$$= \underset{4 \times 4}{P} \bar{x}_w$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} \leftarrow \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$\rightarrow$ last dim still 1
 4×4 4×4

$\rightarrow$ This is omitted because $w=1$ at all times

$$xw = (x \ y \ z \ w)^T$$

$\hookrightarrow$ homogenous

NOTE:

↳ Intrinsic matrix defines projection from camera image plane/sensors.

↳ Extrinsic matrix defines projection from world camera coordinates.

$$\bar{x}_s = \begin{bmatrix} K & 0 \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \bar{x}_w$$

4×4 4×4

$$= \tilde{P} \bar{x}_w$$

4×4

↳ $\bar{x}_s$ is 4D vector & must be normalized w.r.t it's 3rd entry to obtain the inhomogeneous vector

$$\bar{x}_s = \frac{\bar{x}_s}{z_s}$$

$$= (x_s/z_s, y_s/z_s, 1, 1/z_s)^T$$

$\hookrightarrow$ Inverse depth

↳ If inverse depth is known, then we can go from pixel coordinate (2D point) to world coordinate (3D point).

$$\tilde{x}_w = \tilde{P}^{-1} \bar{x}_s$$

Lens Distortion:

↳ Lens are important

↳ With pinhole, very less light is being collected

↳ So, we need lens to collect light but with this geometric distortions are introduced.

↳ There is NO linear projection.

↳ Lines in 3D are projected onto curves in the image plane.

↳ Two types of distortions:

1) Radial

2) Tangential

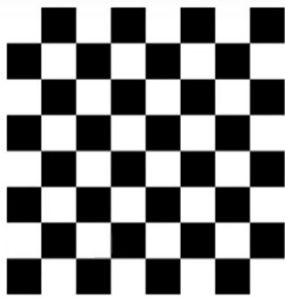
↳ Let $x = \frac{x_c}{z_c}$, $y = \frac{y_c}{z_c}$ & $r^2 = x^2 + y^2$. The distorted point are

$$x' = \underbrace{(1 + k_1 r^2 + k_2 r^4)}_{\text{Radial distortion}} \begin{pmatrix} x \\ y \end{pmatrix} + \underbrace{\begin{pmatrix} 2k_3 xy + k_4 (r^2 + 2x^2) \\ 2k_4 xy + k_3 (r^2 + 2y^2) \end{pmatrix}}_{\text{Tangential Distortion}}$$

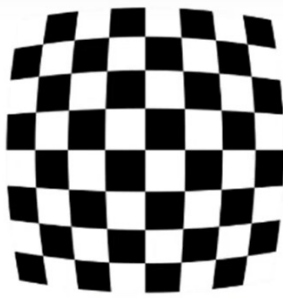
$$x_s = \begin{pmatrix} f_x x' + c_x \\ f_y y' + c_y \end{pmatrix}$$

↳ Images can be undistorted

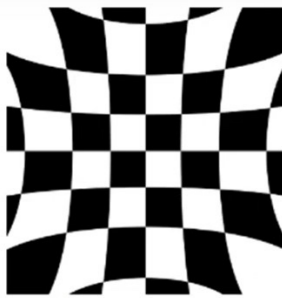
↳ But this can be undone.



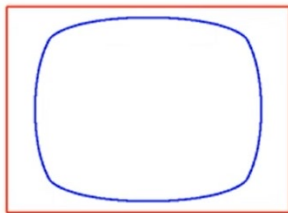
No distortion



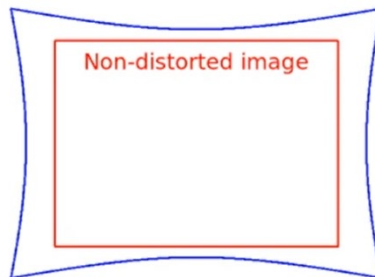
Negative radial distortion
(Barrel distortion)



Positive radial distortion
(Pincushion distortion)



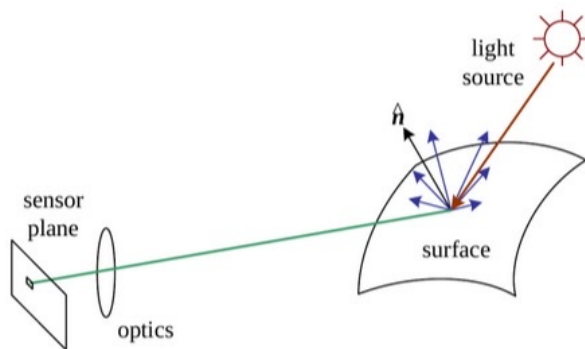
Negative radial distortion ($k_1 = -1.5$)
(Barrel distortion)



Positive radial distortion ($k_1 = 1.5$)
(Pincushion distortion)

Lecture 2.3: Photometric Image Formation

- ↳ How light changes once it reaches the image sensor.
- ↳ How an image is formed in terms of pixel intensities & colors



Rendering Equation: → Describes how light reflects from the surface.

- ↳ Let p be a 3D surface point, $v \in \mathbb{R}^3$ the viewing direction and $r \in \mathbb{R}^3$ the incoming light direction. The rendering equation describes how much of the light L_{in} with wavelength λ arriving at p is reflected into the view direction.

$$L_{out}(p, v, \lambda) = L_{emit}(p, v, \lambda) + \int BRDF(p, r, v, \lambda) \cdot L_{in}(p, r, \lambda) \cdot (n^T r) dr$$

↳ How much light is p emitting?

↳ How light is reflected at point p for given r, v, λ ?

↳ Incoming light

Affect of angle of L_{in} on amount of reflected light at p

↳ $N \parallel r$: max reflection
↳ $N \perp r$: 0

YouTube Sign in

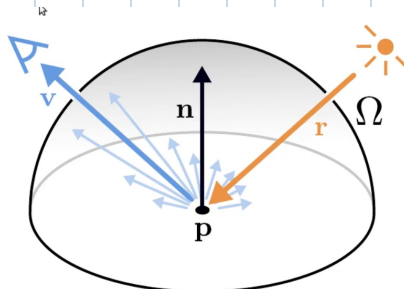


The Rendering Equation

Mark Swaim 11 subscribers

82 likes Share Save

1.6K views 4 years ago
The rendering equation is an equation that simulates how light interacts with a 3D scene. As a result, it can be used to create stunningly realistic artwork. Although the rendering equation can look really complicated at first (and it's actually ...more



NOT 2:

↳ Over the entire hemisphere Consider:

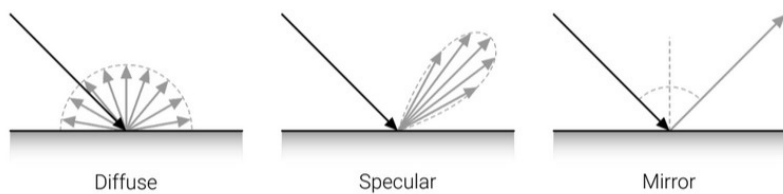
↳ How the available light reflected from p (surface)?

↳ The light source

↳ How much light is available to reflect?

Diffuse and Specular Reflection

- ↳ How do these Bi-directional reflectance distribution functions (BRDF) look like?



↳ Reflected in all directions equally.

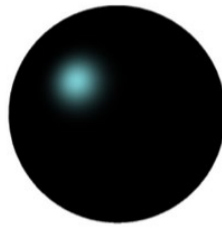
↳ Reflected in a certain direction.

↳ Incoming light is exactly mirrored away into one particular direction.



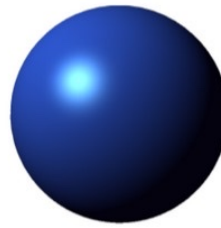
Diffuse

↳ Equally



Specular

↳ In particular direction.

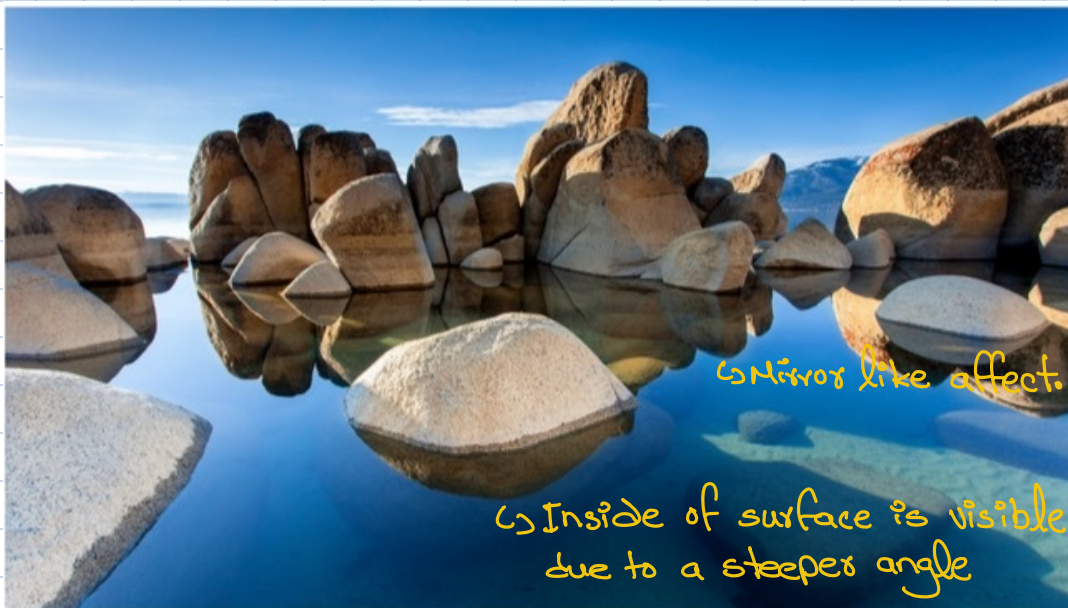


Combined

Fresnel effect:

↳ Take a picture of a lake.

↳ Amount of light reflected depends on the viewing angle.



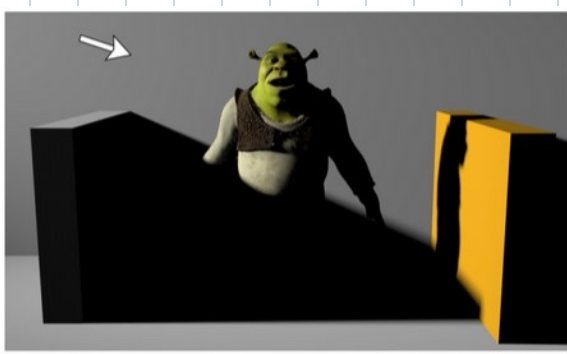
↳ Mirror like effect.

↳ Inside of surface is visible due to a steeper angle

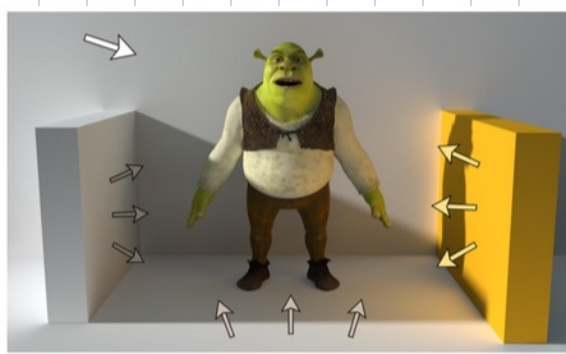
Global illumination

↳ Light might bounce from a surface multiple times.

↳ We need to model this.



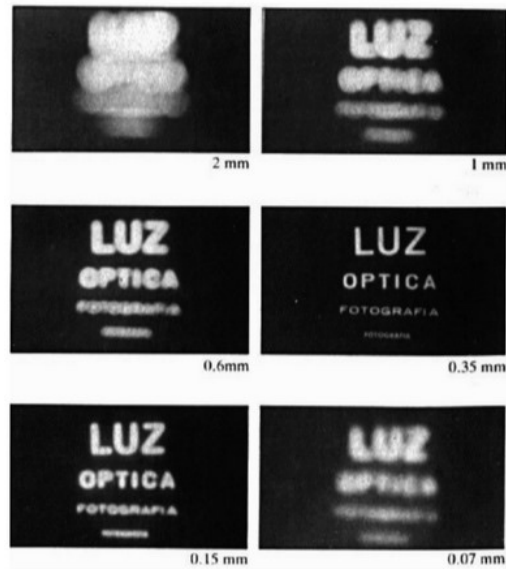
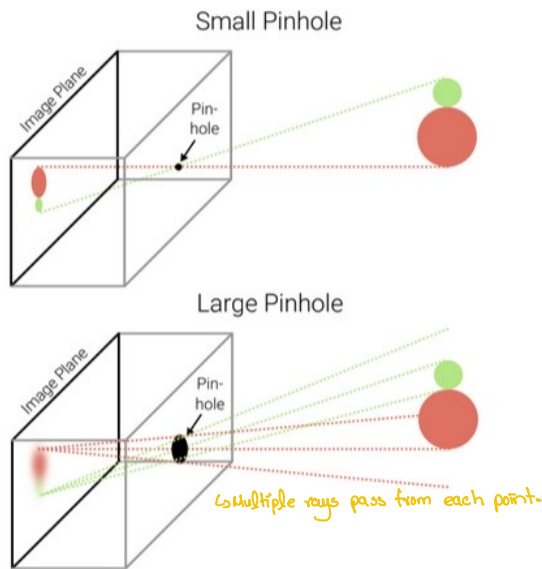
Rendering with Direct Lighting



Rendering with Global Illumination

Why use Camera lens?

↳ Large & very small result in blur images

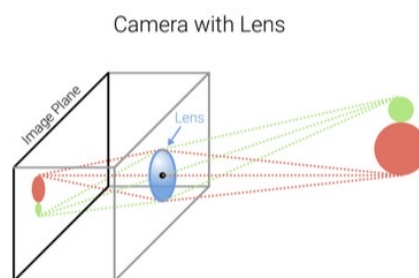
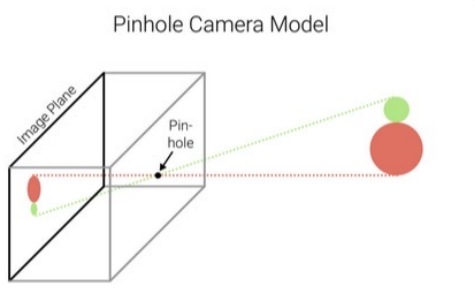


↳ starts diffracting light.
↳ Bend around edges.

Optics:

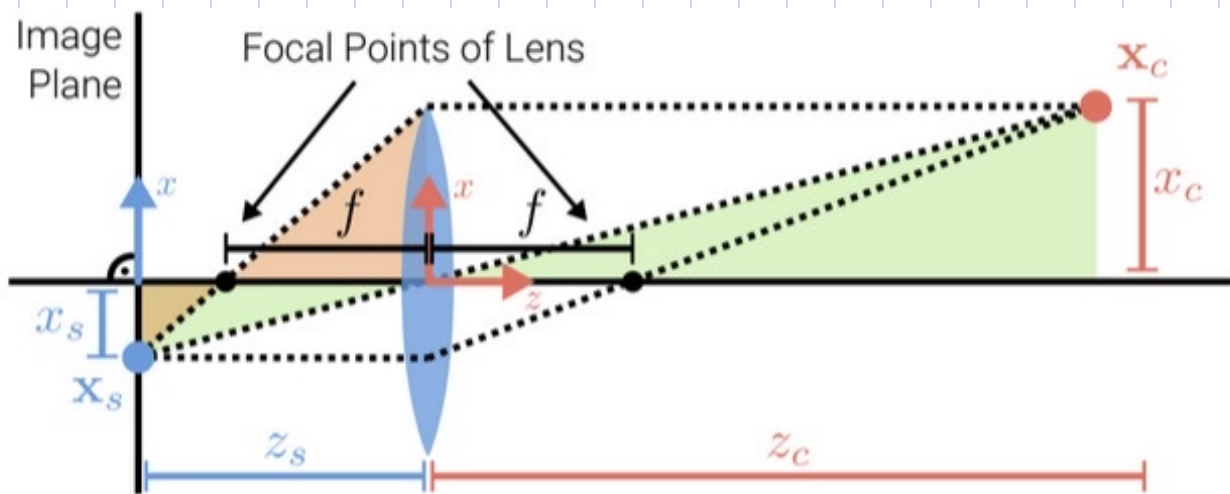
Think funnel that focuses water direction

↳ Camera uses multiple lens or one to collect light on the image plane.



↳ For many applications, it suffices to model cameras with pinhole model
↳ However, to address focus, vignetting & aberration we need to model lenses. Simplest being:

Thin Lens Model



↳ Light from a single point all focus at one point.

Green triangles:

$$\frac{x_s}{z_s} = \frac{x_c}{z_c}$$

$$\frac{x_s}{x_c} = \frac{z_s}{z_c}$$

Red triangles:

$$\frac{x_s}{z_s - f} = \frac{x_c}{f}$$

$$\frac{x_s}{x_c} = \frac{z_s - f}{f}$$

$$\hookrightarrow \frac{z_s}{z_c} = \frac{z_s - f}{f}$$

$$= \frac{z_s}{f} - 1$$

$$\frac{1}{f} = \frac{1}{z_s} + \frac{1}{z_c} \quad \left\{ \begin{array}{l} \text{image in focus if this} \\ \text{relationship holds} \end{array} \right.$$

↳ For a particular 3D-point x_c & distance to image plane z_s , we can determine how we have change f (focal length) such that a point appears sharply.

Depth of image (DoF)

↳ Another way to control circle of confusion is to change aperture

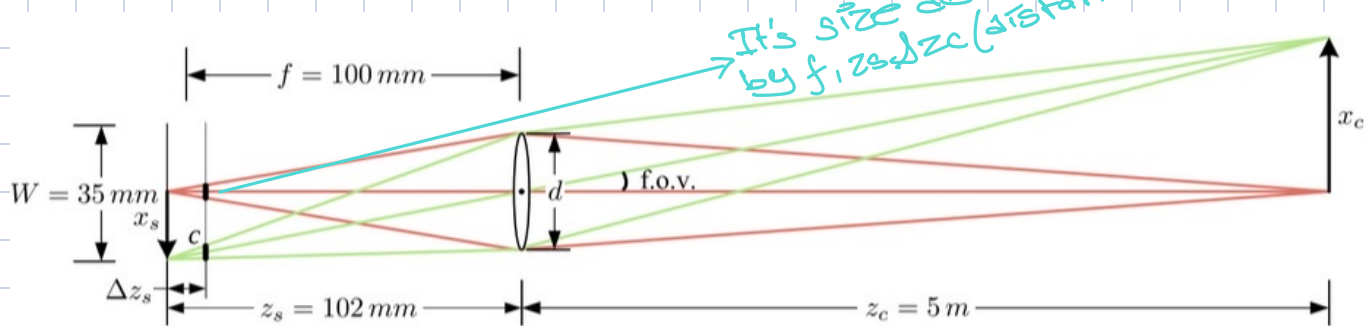
↳ The range where image appears sharply is called DoF

↳ For $z_c \rightarrow \infty$, $z_s = f$ (lens with $f \approx$ pinhole at distance f)

↳ If image plane is out of focus, 3D point projects to a circle of confused $\odot$ c.

↳ If pixel size $> c$, we get blur.

(determined by distance of 3D point)



→ Images are sharper but noisy

↳ A smaller aperture (A covers over lens) limits amount of light passed.

↳ Aperture in DSLR indicated by f number;

$$N = \frac{f}{d} \rightarrow \text{focal length}$$

↳ Aperture diameter



Aperture = $f/1.4$
DOF = 0.8 cm

↳ Blurs over distance
due small aperture



Aperture = $f/4.0$
DOF = 2.2 cm



Aperture = $f/22$
DOF = 12.4 cm

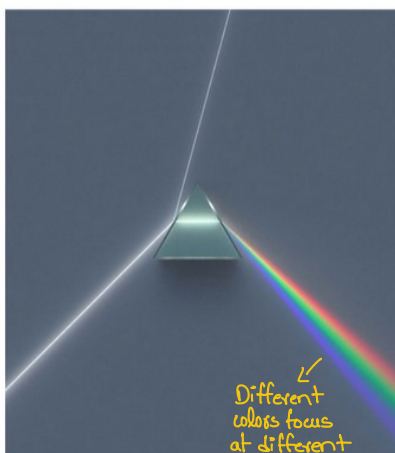
↳ Sharpe & almost no blur
due to large aperture.

Chromatic Aberration:

↳ Tendency of light different colors to focus at slightly different distances.

↳ Red color focus after image plane

↳ Blue color focus in front of image plane.



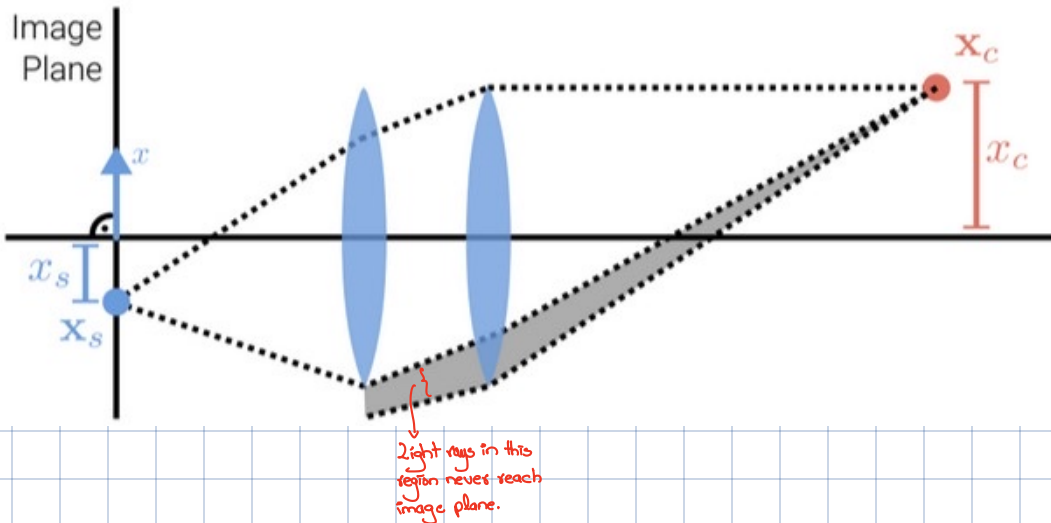
↙ Different colors focus at different points.



↳ Chromatic aberration

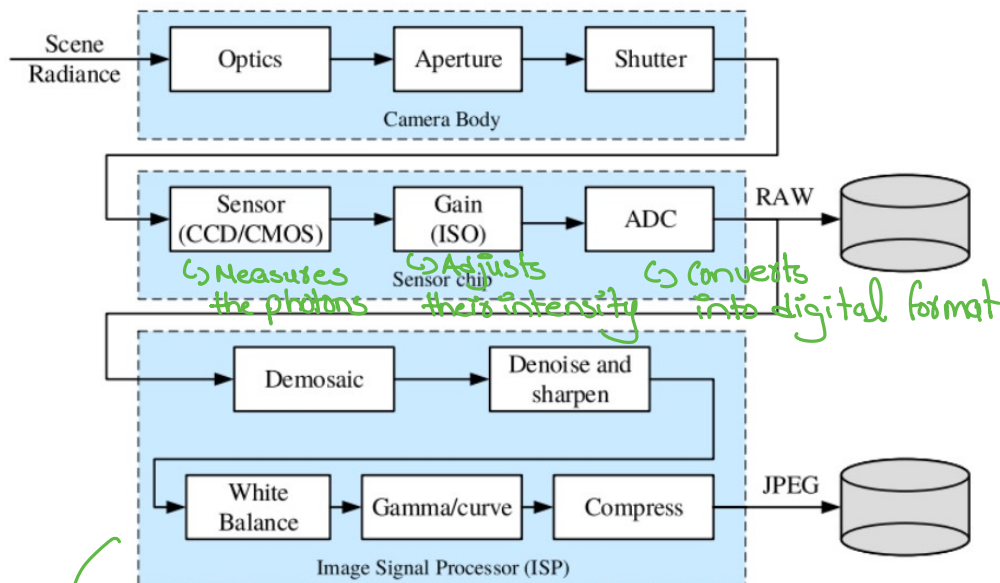
Vignetting:

- ↳ Tendency for brightness to fall off towards the image edge.
- ↳ Combination of:
 - ↳ Natural vignetting: foreshortening of object & aperture
 - ↳ Mechanical vignetting: The shaded part of beam never reaching image.



Lecture 2.4: Image Sensing Pipeline

- ↳ What happens to light after it has passed through the lens and arrived on the image sensors until it's stored in a digital format?



- ↳ Denoised
- ↳ Colors adjusted
- ↳ White balance adjusted
- ↳ Compressed.

Shutter:

- ↳ Just before light hits the image sensor, there is a shutter.
- ↳ Open for a fraction of time
- ↳ The shutter speed (exposure time) controls how much light reaches the sensor.
- ↳ Determines if image is Over OR under exposed

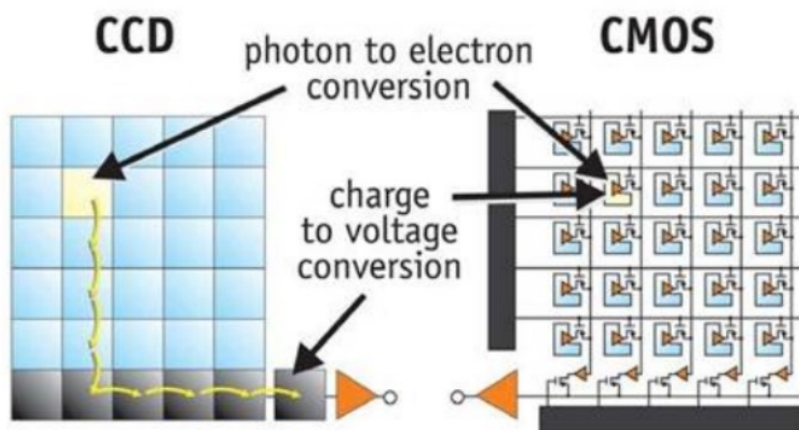
- | | |
|--|-------------------------------|
| ↓ | ↓ |
| ↳ Shutter open for too long | ↳ Shutter open for too small. |
| ↳ Bright/light spots | ↳ Dark spots |
| ↳ For Dark sensor, open shutter for long time. | |

- ↳ For Dark sensor,
To avoid motion blur, you keep exposure time small then you don't suffer from blur but you increase the noise because there are less photons per pixel

Convert light to Digital image

Sensor:

- ↳ How light is measured?
- ↳ CCDs move charge from pixel to pixel and convert it to voltage at the output node.
- ↳ CMOS images convert charge to voltage inside each pixel & are standard.
- ↳ After light/scene is captured on the image sensor, it is then converted into voltage for processing.



Color Filter Analysis:

- ↳ In order to take pictures of color we have to measure color
- ↳ Color is measured by putting a little filter of color on top of each pixel.
- ↳ Because each pixel can only measure one color, so pixels are arranged in a certain pattern called color array.
- ↳ Each pixel in the color array is responsive to a particular color.
- ↳ Most common pattern is Bayer RHB pattern
- ↳ Missing color at each pixel is interpolated from the neighbours called Demosaicing.
 - ↳ Educated guess

G	R	G	R
B	G	B	G
G	R	G	R
B	G	B	G

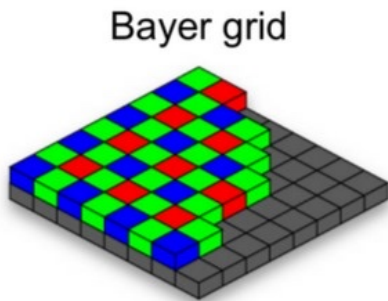
Bayer RGB Pattern

rGb	Rgb	rGb	Rgb
rgB	rGb	rgB	rGb
rGb	Rgb	rGb	Rgb
rgB	rGb	rgB	rGb

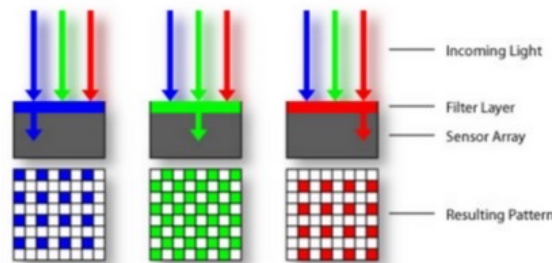
Interpolated Pixels

NOTE:

- ↳ A point on object might be of color that has certain RHB values
- ↳ A single sensor captures only one of these values.
- ↳ Thus, we can't get the exact color.
- ↳ So, we use above techniques.



Bayer grid



NOTE:

- ↳ Sensor itself doesn't itself know which color it's capturing.
- ↳ It only knows the intensity of light.

- ↳ After we get one color values at each pixel locations.
- ↳ The pixel are converted into voltages & then missing color values are estimated from surrounding voltages.

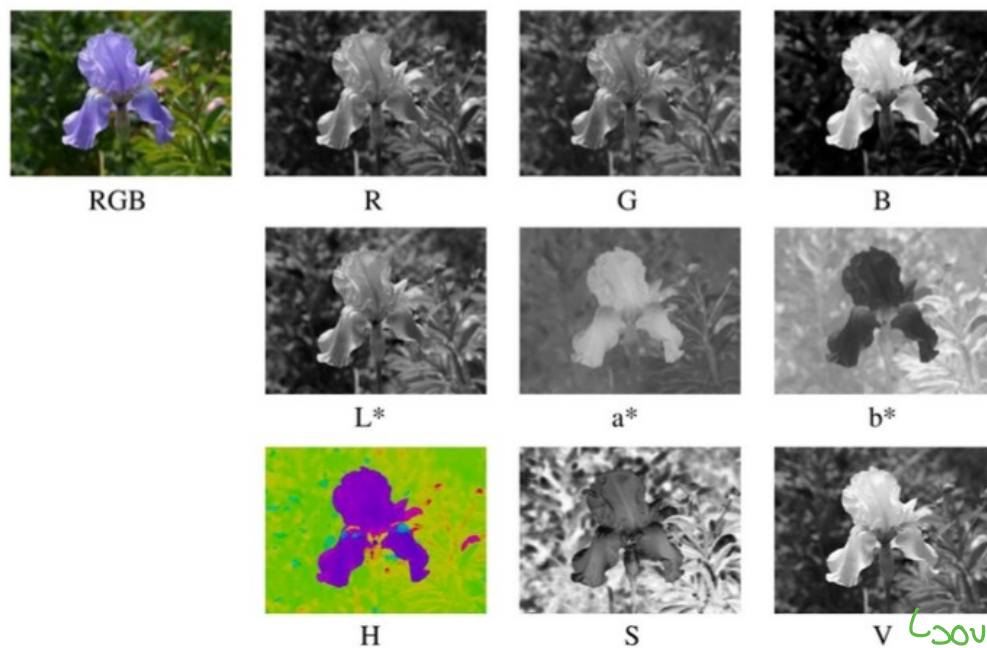
↳ Individual pixels don't record a single wavelength but rather entire range of wavelength or green for example

↳ Each pixel integrates the light spectrum response curve L according to its / camera's spectral sensitivity S .

$$R = \int L(\lambda) S_R(\lambda) d\lambda$$

↳ How well a camera capture wavelengths of different colors?

Color spaces:



↳ Separating in Hue, saturation
 & value

Gamma Compression:

- ↳ To stored images on disk we have to discretize the values.
- ↳ Since, humans are more sensitive intensity differences in darker regions.
- ↳ Gamma compression non-linearly transforms (left) the intensities or colors prior to discretization (left) and to undo this transformation during loading.
- ↳ This keeps the intensity differences in dark regions intact.

Image Compression:

- ↳ Luminance is compressed with higher fidelity than chrominance.
- ↳ Often (8x8) patch based discrete cosine or wavelet transformations are used.
- ↳ Discrete Cosine Transform (DCT) $\approx$ PCA
- ↳ Coefficients are stored with Huffman codes.